확인 질문과 연습 문제 앞의 체크박스(☐)를 클릭하면 공부한 항목을 표시할 수 있습니다. 저장(Ctrl+S)하면 남습니다.

코드 분석 공부 자료 - 조각김치 이물검출 라벨링 프로그램

> 이 자료는 우리 팀 최종 프로그램(`kimchi_labeler`)의 코드를 **파일 순서대로 읽으면서 해석**하는 공부용 자료입니다. VS Code 에서 이 파일을 열고 `Ctrl+Shift+V` 를 누르면 보기 좋게 표시됩니다. 왼쪽에 해당 코드 파일을 같이 열어 두고 읽으세요. 각 장 끝의 **확인 질문**에 답할 수 있으면 그 파일은 이해한 것입니다. 발표 때 받을 만한 질문도 마지막 장에 모았습니다.

목차

0. 먼저 큰 그림
1. config.py - 팀 기준을 코드로 고정
2. yolo/yolo_io.py - YOLO TXT 읽기·쓰기와 좌표 변환
3. bbox/view.py - 화면 좌표와 원본 좌표
4. yolo/dataset.py - RAW 폴더 구조 조사
5. manifest.py - 900장의 작업 장부
6. bbox/session.py - 편집과 저장의 두뇌
7. validation/validator.py - 자동 검사
8. ui/ - 화면, 마우스, 메뉴
9. scripts - 운영 도구 00~10
10. tests - 자동 테스트
11. 따라가 보기: Space 를 눌렀을 때 일어나는 일
12. 연습 문제
13. 발표 예상 질문과 답

---

0. 먼저 큰 그림

0.1 이 프로그램이 하는 일

```
RAW (회사 원본 JPG + YOLO TXT, 읽기만)
   |  프로그램이 TXT 를 읽어 BBox 를 화면에 그림
   ▼
사람이 검수: 맞으면 PASS / 고치면 EDITED / 애매하면 REVIEW
   |  저장
   ▼
WORK (수정한 TXT + manifest.csv 작업 장부)
   |  자동 Validation + 교차검수 + PASS 표본검수
   ▼
FINAL (조건을 통과해야만 생성, 교과 8 로 인계)
```

0.2 파일 지도와 역할 담당

파일	한 줄 설명	담당
`main.py`	프로그램 시작점. 명령줄 인자를 받아 `ui/app.py` 의 `run()` 호출	PM
`setup_wsl.sh` , `run.sh`	WSL 처음 설치(패키지·글꼴·.venv·테스트) / 실행	PM
`configs/classes.yaml`	Class 0~6 이름·사용 여부·화면 색 (프로그램이 읽는 설정 파일)	PM
`src/config.py`	경로, 상태 목록 같은 **팀 기준**, classes.yaml 읽기	PM
`src/yolo/yolo_io.py`	YOLO TXT ↔ Box 변환, 안전 저장	좌표·저장
`src/bbox/view.py`	화면 픽셀 ↔ 원본 픽셀 변환, Zoom/Pan	좌표
`src/yolo/dataset.py`	RAW 폴더를 뒤져 이미지 목록과 출처 정보 만들기	저장·Data
`src/manifest.py`	manifest.csv 읽기·쓰기·병합	Data
`src/manifest_builder.py`	새 이미지를 Manifest 에 PENDING 으로 추가	Data
`src/bbox/session.py`	불러오기, BBox 편집, 저장 규칙 (**GUI 없는 핵심 로직**)	저장
`src/validation/validator.py`	형식 오류 자동 검사	QA
`src/validation/stats.py`	추가/삭제/Class 변경 통계	Data
`src/yolo/final_layout.py`	FINAL `images/` · `labels/` 파일 이름 규칙	PM
`src/ui/app.py`	메인 창 `LabelApp` : 메뉴 구성, 단축키, 이미지 이동, 저장, 통계	GUI
`src/ui/panels.py`	위젯 배치만 (왼쪽 목록, Canvas, 오른쪽 두 열, 아래 도구 막대)	GUI
`src/bbox/canvas_editor.py`	이미지 그리기, BBox 그리기, 마우스 편집	GUI·좌표
`src/ui/menubar.py`	직접 만든 메뉴 막대 (WSLg 잔상 문제 해결)	GUI
`src/ui/dialogs.py`	Validation 결과 표, 진행현황, Class 통계, 단축키 안내 창	GUI·QA
`src/ui/style.py`	색, 글꼴, 상태 기호, 버튼 만드는 함수	GUI
`scripts/00~10`	스냅샷, Manifest, 검사, 배분, 병합, QA, FINAL, Handoff, Golden, Manifest 내보내기	역할별
`tests/test_core.py`	전체 흐름 자동 테스트	QA

0.3 가장 중요한 설계 원칙 3가지

1. **화면(GUI)과 데이터 로직(session)을 분리했다.** `ui/` 는 "무엇을 보여줄까"만, `session.py` 는 "데이터를 어떻게 바꾸고 저장할까"만 담당합니다. 그래서 창을 띄우지 않고도 저장·좌표 로직을 자동 테스트할 수 있습니다.
2. **BBox 는 항상 원본 이미지 픽셀 좌표로 들고 있는다.** 화면에 그릴 때만 변환합니다. 그래서 Zoom/Pan 을 아무리 해도 BBox 의 실제 위치가 흔들리지 않습니다.
3. **RAW 는 절대 쓰지 않는다.** 저장은 WORK 로만 가고, 저장 직전에 경로를 한 번 더 검사하며, SHA-256 해시로 RAW 가 바뀌지 않았음을 증명합니다.

0.4 실행 흐름 (bash run.sh 부터 창이 뜰 때까지)

```
run.sh  -> .venv 켜기 -> python main.py "$@"
main.py
  └ run()                              ui/app.py 맨 아래
     ├ ask_worker()   : settings.json 의 작업자 이름 (없으면 물어보고 저장)
     ├ ask_session()  : --raw 가 있으면 그 폴더, 없으면 settings 의 raw_root 로 LabelSession
     |    |              (WORK = data/raw 면 data/work, 그 밖에는 data/work_<RAW폴더이름>. settings.json 에 기억)
     |    ├ scan_images(raw)  : RAW 의 이미지 목록
     |    └ Manifest(...)     : 없는 이미지는 PENDING 으로 추가
     |   (실패하면 폴더 선택 창 -> open_session())
     ├ LabelApp(root, session)  : build_menu() -> build_main_window() -> bind_canvas() -> bind_keys()
     └ root.mainloop()          : 이벤트(클릭, 키) 기다리기
```

---

1. config.py - 팀 기준을 코드로 고정

Class 표는 코드가 아니라 **설정 파일** `configs/classes.yaml` 에 있고, `config.py` 가 프로그램 시작 때 읽습니다.

```yaml
classes:
  0:
    name: "나뭇잎 · 종이류"
    enabled: true
    color: "#2ecc71"
  ...
  4:
    name: "고무장갑(미사용)"
    enabled: false        # 사용하지 않는 Class
    color: "#ff0000"
```

```
CLASSES = load_classes()                    # configs/classes.yaml 읽기
CLASS_NAMES = {cid: c["name"] for cid, c in CLASSES.items()}
CLASS_COLORS = {cid: c["color"] for cid, c in CLASSES.items()}
VALID_CLASS_IDS = set(CLASS_NAMES)                                  # 0~6 : 형식상 허용
UNUSED_CLASS_IDS = {cid for cid, c in CLASSES.items() if not c["enabled"]}   # {4}
ACTIVE_CLASS_IDS = sorted(VALID_CLASS_IDS - UNUSED_CLASS_IDS)
```

해석

- Class 기준을 **설정 파일 한 곳**에 모았습니다. 다른 파일은 숫자를 직접 쓰지 않고 `CLASS_NAMES` 같은 상수를 import 해서 씁니다. 이름이나 색을 바꿀 때 코드를 고치지 않고 yaml 만 고치면 됩니다.
- `yaml.safe_load` 는 YAML 을 파이썬 딕셔너리로 바꿔 줍니다. `safe_` 는 파일 안에 코드를 실행하는 내용이 있어도 실행하지 않는 안전한 방식입니다.
- `VALID` (0~6, TXT 에 있어도 형식 오류는 아님)와 `ACTIVE` (새 BBox 에 쓸 수 있음, 4 제외)를 나눈 것이 핵심입니다. `enabled: false` 인 4번은 기존 TXT 에서는 **지우지 않고 표시**만 하고, 새로 만드는 것은 막습니다.
- `VALID_CLASS_IDS - UNUSED_CLASS_IDS` 는 집합 차집합입니다. 결과는 `[0, 1, 2, 3, 5, 6]` 입니다.

```
class Paths:
    def check_not_same(self) -> None:
```

```
        raw, work = self.raw.resolve(), self.work.resolve()
        if raw == work or raw in work.parents or work in raw.parents:
            raise SystemExit(f"[중단] RAW({raw}) 와 WORK({work}) 는 서로 다른 독립 폴더여야 합니다.")
```

해석

- `resolve()` 는 상대경로·`..` 를 풀어 **진짜 절대경로**로 만듭니다.
- `work.parents` 는 WORK 의 모든 상위 폴더 목록입니다. `raw in work.parents` 가 참이면 WORK 가 RAW 안에 있다는 뜻이고, 그러면 저장할 때 RAW 를 건드릴 위험이 있으므로 시작부터 막습니다.
- 처음에 RAW 로 프로그램 폴더(`group2_project`)를 잘못 골랐을 때 `[중단]` 메시지가 뜬 이유가 바로 이 함수입니다.

확인 질문

- ☐ Class 4 를 `classes.yaml` 에서 아예 지우면 어떤 문제가 생길까요? (힌트: 기존 TXT 의 4 가 "범위 밖 오류"로 바뀝니다)
- ☐ `settings.json` 을 Git 에 올리지 않는 이유는 무엇일까요?

---

2. yolo/yolo_io.py - YOLO TXT 읽기·쓰기와 좌표 변환

2.1 Box: 프로그램 안의 BBox 표현

```
@dataclass
class Box:
    """원본 이미지 픽셀 좌표 BBox."""
    cls: int
    x1: float
    y1: float
    x2: float
    y2: float

    def normalized(self) -> "Box":
        """x1<x2, y1<y2 가 되도록 정렬한 복사본."""
        return Box(self.cls, min(self.x1, self.x2), min(self.y1, self.y2),
                   max(self.x1, self.x2), max(self.y1, self.y2))

    def clamp(self, img_w: int, img_h: int) -> "Box":
        b = self.normalized()
        return Box(b.cls, max(0.0, min(b.x1, img_w)), max(0.0, min(b.y1, img_h)),
                   max(0.0, min(b.x2, img_w)), max(0.0, min(b.y2, img_h)))
```

해석

- `@dataclass` 는 `__init__` 을 자동으로 만들어 줍니다. `Box(2, 10, 20, 110, 80)` 처럼 바로 만들 수 있습니다.
- YOLO 는 (중심, 너비, 높이) 형식이지만, 프로그램 안에서는 **왼쪽 위 (x1,y1) ~ 오른쪽 아래 (x2,y2)** 로 들고 있습니다. 클릭 판정, 그리기, 크기 조절이 훨씬 쉽기 때문입니다.
- 마우스를 오른쪽 아래에서 왼쪽 위로 드래그하면 x1 > x2 가 됩니다. `normalized()` 가 min/max 로 순서를 바로잡습니다.
- `clamp()` 는 이미지 밖으로 나간 부분을 잘라냅니다. `max(0, min(값, 최대))` 는 "0 과 최대 사이로 가두기"의 정석 패턴입니다.

2.2 YOLO ↔ 픽셀 변환 (가장 중요한 공식)

```
def yolo_to_pixel(cls, cx, cy, w, h, img_w, img_h) -> Box:
    x1 = (cx - w / 2.0) * img_w
    y1 = (cy - h / 2.0) * img_h
    x2 = (cx + w / 2.0) * img_w
    y2 = (cy + h / 2.0) * img_h
    return Box(cls, x1, y1, x2, y2)


def pixel_to_yolo(box, img_w, img_h):
    b = box.normalized()
    cx = (b.x1 + b.x2) / 2.0 / img_w
    cy = (b.y1 + b.y2) / 2.0 / img_h
    w = (b.x2 - b.x1) / img_w
    h = (b.y2 - b.y1) / img_h
    return b.cls, cx, cy, w, h
```

손으로 계산해 보기 (4K 이미지 3840×2160, TXT `3 0.5 0.5 0.1 0.2`)

값	계산	결과
x1	(0.5 − 0.05) × 3840	1728
x2	(0.5 + 0.05) × 3840	2112
y1	(0.5 − 0.1) × 2160	648
y2	(0.5 + 0.1) × 2160	1512

다시 YOLO 로: cx = (1728+2112)/2/3840 = 0.5, w = (2112−1728)/3840 = 0.1 → 원래 값으로 돌아옵니다. 이것이 **Round-trip(왕복) 검사**이고, `09_golden_test.py` 가 실제 데이터로 오차 < 0.000001 인지 확인합니다.

2.3 한 줄 파싱: 자동으로 고치지 않고 오류만 알려준다

```
def parse_line(line: str):
    parts = line.split()
    if len(parts) != 5:
        return None, f"값 개수 {len(parts)}개 (5개여야 함)"
    try:
        cls_f = float(parts[0])
    except ValueError:
        return None, f"class 숫자 변환 실패 '{parts[0]}'"
    if not cls_f.is_integer():
        return None, f"class 가 정수가 아님 '{parts[0]}'"
    try:
        cx, cy, w, h = (float(p) for p in parts[1:])
    except ValueError:
        return None, "좌표 숫자 변환 실패"
    return (int(cls_f), cx, cy, w, h), None
```

해석

- 반환값이 `(값, 오류)` **두 개짜리 튜플**입니다. 성공하면 `(값, None)`, 실패하면 `(None, 메시지)`. 호출한 쪽은 `val, err = parse_line(line)` 으로 받아 `if err:` 로 분기합니다.
- `float("1.0").is_integer()` 는 True 이므로 `1.0` 도 Class 1 로 받아주고, `1.5` 는 오류로 처리합니다.
- 예외(`try/except`)를 프로그램 밖으로 던지지 않고 메시지로 바꿔 돌려주므로, TXT 한 줄이 깨져도 프로그램이 죽지 않습니다.

2.4 안전 저장 (atomic write)

```
def atomic_write_text(path: Path, text: str) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    fd, tmp = tempfile.mkstemp(dir=str(path.parent), prefix=".tmp_", suffix=path.suffix)
    try:
        with os.fdopen(fd, "w", encoding="utf-8", newline="\n") as f:
            f.write(text)
        os.replace(tmp, path)
    except BaseException:
        if os.path.exists(tmp):
            os.remove(tmp)
        raise
```

해석

- 원본 파일에 바로 쓰다가 컴퓨터가 꺼지면 **반쯤 쓴 깨진 TXT** 가 남습니다.
- 그래서 ① 같은 폴더에 임시파일을 만들고 ② 다 쓴 다음 ③ `os.replace` 로 **한 번에 바꿔치기** 합니다. `os.replace` 는 운영체제가 "전부 아니면 전무"로 처리하므로, 결과는 옛 파일이거나 새 파일 둘 중 하나입니다.
- `atomic_copy()` 는 같은 방식으로 **바이트 그대로 복사**합니다. 라벨을 안 바꾼 경우 RAW TXT 를 이걸로 복사해서, 줄바꿈(CRLF)이나 소수점 자릿수까지 원본과 똑같이 유지합니다.

확인 질문

- ☐ TXT 가 `2 1.2 0.5 0.1 0.1` 이면 `parse_line` 은 성공할까요? 그럼 이 오류는 어디서 잡힐까요? (→ 7장 validator)
- ☐ 왜 Box 를 YOLO 형식(중심/너비)으로 들고 있지 않을까요?

---

3. bbox/view.py - 화면 좌표와 원본 좌표

```
class ViewTransform:
    def to_canvas(self, x, y):
        return x * self.scale + self.offset_x, y * self.scale + self.offset_y

    def to_image(self, cx, cy):
        return (cx - self.offset_x) / self.scale, (cy - self.offset_y) / self.scale
```

해석

- 원본 이미지의 점 (x, y) 를 화면에 그리려면 `배율(scale)` 을 곱하고 `이동량(offset)` 을 더합니다. 반대는 빼고 나눕니다. **이 두 줄이 Zoom/Pan 의 전부**입니다.
- 4K 이미지를 1200px 화면에 맞추면 scale ≈ 0.3 입니다. 화면에서 1px 움직이면 원본에서는 약 3.3px 움직인 것입니다.

3.1 Fit: 이미지 전체를 화면에 맞추기

```
def fit(self, img_w, img_h, canvas_w, canvas_h, margin=10):
    cw, ch = max(canvas_w - 2 * margin, 1), max(canvas_h - 2 * margin, 1)
    self.scale = max(MIN_SCALE, min(cw / img_w, ch / img_h))
    self.offset_x = (canvas_w - img_w * self.scale) / 2.0
    self.offset_y = (canvas_h - img_h * self.scale) / 2.0
```

- 가로 비율과 세로 비율 중 **작은 쪽**을 고르면 이미지가 잘리지 않고 다 들어갑니다.
- offset 은 남는 공간의 절반, 즉 **가운데 정렬**입니다.

3.2 마우스 위치 기준 Zoom

```
def zoom_at(self, factor, cx, cy):
    ix, iy = self.to_image(cx, cy)          # ① 마우스 아래 원본 좌표 기억
    new_scale = max(MIN_SCALE, min(MAX_SCALE, self.scale * factor))
    self.scale = new_scale                   # ② 배율 변경
    self.offset_x = cx - ix * new_scale      # ③ 그 원본 점이 다시 마우스 아래 오도록 offset 계산
    self.offset_y = cy - iy * new_scale
```

- ③ 은 `to_canvas` 공식 `cx = ix * scale + offset` 을 offset 에 대해 푼 것입니다.
- 이렇게 하면 휠을 굴려도 **마우스가 가리키는 이물이 화면에서 도망가지 않습니다**. 작은 이물을 찾을 때 매우 중요합니다.

확인 질문

- [ ] scale = 0.5, offset = (100, 50) 일 때 원본 (400, 300) 은 화면 어디에 그려질까요? (답: (300, 200))
- [ ] Zoom 후에도 BBox 가 객체에서 벗어나지 않는 이유를 이 파일로 설명해 보세요.

---

4. yolo/dataset.py - RAW 폴더 구조 조사

```
def image_rel_to_label_rel(rel_image: str) -> tuple[str, str, str]:
    """이미지 상대경로 -> (라벨 상대경로, source_dataset, original_split)."""
    parts = list(PurePosixPath(rel_image).parts)
    stem_txt = str(PurePosixPath(parts[-1]).with_suffix(".txt"))
    if "images" in parts[:-1]:
        idx = len(parts) - 2 - parts[:-1][::-1].index("images")   # 마지막 'images'
        label_parts = parts[:idx] + ["labels"] + parts[idx + 1:-1] + [stem_txt]
        source = parts[idx - 1] if idx >= 1 else "root"
        split = parts[idx + 1] if idx + 1 < len(parts) - 1 else "unknown"
    ...
```

예시로 따라가기: `이물검출_학습데이터2/images/validation/a.jpg`

변수	값
parts	`['이물검출_학습데이터2', 'images', 'validation', 'a.jpg']`
idx (images 위치)	1
label_parts	`['이물검출_학습데이터2', 'labels', 'validation', 'a.txt']`

변수	값
source	`이물검출_학습데이터2` (images 바로 앞 폴더)
split	`validation` (images 바로 뒤 폴더)

해석

- 폴더 경로 자체에서 **출처(Dataset 1/2)와 원래 split(train/validation)** 을 뽑아냅니다. 가이드가 강조한 "출처 정보 보존"을 코드로 구현한 부분입니다.
- 이미지를 구분하는 키는 파일명이 아니라 **RAW 기준 상대경로**(`rel_image`)입니다. Dataset 1 과 2 에 같은 파일명이 있어도 섞이지 않습니다.
- `effective_label()` 은 "WORK 에 저장본이 있으면 그것, 없으면 RAW" 를 돌려줍니다. 프로그램 전체가 이 규칙 하나로 '현재 라벨'을 결정합니다.

확인 질문

☐ `scan_orphan_labels()` 는 무엇을 찾나요? 왜 `classes.txt` 는 제외할까요?

---

5. manifest.py - 900장의 작업 장부

```
COLUMNS = [
    "relative_path", "image_name", "label_name", "label_relative_path",
    "source_dataset", "original_split", "scene_type",
    "assignee", "reviewer", "status",
    "original_bbox_count", "final_bbox_count",
    "issue", "note", "pilot", "pass_sample", "updated_at", "updated_by",
]
```

해석

- 가이드 5.2장의 Manifest 최소 필드에 `pilot` , `pass_sample` , `updated_at` , `updated_by` 를 더했습니다. 마지막 두 개는 **병합할 때 누구의 기록이 최신인지** 판단하는 데 씁니다.
- CSV 는 `utf-8-sig` (BOM 포함)로 저장해서 엑셀로 열어도 한글이 깨지지 않습니다.

```
def _rank(row: dict) -> tuple:
    """최신 기록 우선. 같은 시각이면 사람이 작업한 기록이 배분/생성 기록보다 우선."""
    return row["updated_at"], row["updated_by"] not in ("", "builder", "assign")


def merge_manifests(paths):
    merged, origin = {}, {}
    for p in paths:
        m = Manifest(p)
        for key, row in m.rows.items():
            cur = merged.get(key)
            if cur is None or _rank(row) > _rank(cur):
                merged[key] = row
                origin[key] = p
    return merged, origin
```

해석

- 6명이 각자 PC 에서 작업한 manifest 를 합칠 때, 같은 이미지는 **updated_at 이 가장 늦은 행**을 씁니다.
- 날짜 문자열 `"2026-10-14 15:30:02"` 은 **문자열 비교 = 시간 비교**가 되도록 큰 단위부터 썼습니다.
- 튜플 비교는 첫 값이 같으면 두 번째 값을 비교합니다. 그래서 같은 초에 기록됐다면 사람(True)이 스크립트(False)를 이깁니다. 실제로 테스트에서 이 경우가 나와서 추가한 코드입니다.
- `origin` 에는 "이 행을 어느 팀원 폴더에서 가져왔는지"를 기억해서, `04_merge.py` 가 라벨 TXT 도 같은 사람 폴더에서 복사합니다.

확인 질문

☐ 담당자 A 가 15:00 에 EDITED 로 저장하고, 검수자 B 가 16:00 에 REVIEWED 로 저장했습니다. 병합 결과는 누구의 행일까요?

---

6. bbox/session.py - 편집과 저장의 두뇌

6.1 LoadedImage: 지금 열린 이미지 한 장의 상태

```
@dataclass
class LoadedImage:
    rec: ImageRecord
    width: int
    height: int
    boxes: list[Box]                    # 화면에서 편집 중인 BBox
    raw_boxes: list[Box]                # RAW 원본 (변경 여부 비교용)
    raw_text: str ¦ None                # RAW TXT 원문 (없으면 None)
    errors: list[str] = field(default_factory=list)
    from_work: bool = False
    raw_had_errors: bool = False
    undo_stack: list[list[Box]] = field(default_factory=list)
    loaded_snapshot: list[str] = field(default_factory=list)
```

- `boxes` 와 `raw_boxes` 를 **따로** 들고 있어서, 언제든 "RAW 와 달라졌나?"를 비교할 수 있습니다.
- `field(default_factory=list)` 는 객체마다 **새 리스트**를 만들어 줍니다. `= []` 로 쓰면 모든 객체가 리스트 하나를 공유하는 유명한 버그가 생깁니다.

6.2 변경 여부 판단: canonical

```
def canonical(boxes, w, h) -> list[str]:
    """비교용 문자열 목록 (순서 무관)."""
    return sorted(format_line(*pixel_to_yolo(b, w, h)) for b in boxes)
```

- 소수점 6자리 YOLO 문자열로 바꾼 뒤 **정렬**합니다. BBox 순서만 바뀐 것은 "변경 없음"으로, 0.0000001 같은 미세한 실수 오차도 무시하게 됩니다.
- `changed_since_load()` 는 "열었을 때와 비교" → 저장 안 됨(●) 표시와 이동 경고에 쓰입니다.
- `changed_vs_raw()` 는 "RAW 와 비교" → PASS/EDITED 자동 판정에 쓰입니다.

6.3 load(): 이전 이미지 상태를 완전히 버린다

```
def load(self, index: int) -> LoadedImage:
    """index 이미지를 불러온다. 이전 이미지 상태는 완전히 버린다 (BBox 잔상 방지)."""
    self.index = max(0, min(index, self.total - 1))
    rec = self.by_key[self.view_keys[self.index]]
    w, h, _ = image_size(rec.raw_image(self.paths))
    raw_path = rec.raw_label(self.paths)
    raw_res = read_label_file(raw_path, w, h)
    ...
    work_path = rec.work_label(self.paths)
    if work_path.exists():
        res = read_label_file(work_path, w, h)
        from_work = True
    else:
        res = raw_res
        from_work = False
    boxes = [copy.copy(b) for b in res.boxes]
    self.cur = LoadedImage(rec, w, h, boxes, raw_res.boxes, raw_text, ...)
```

- 가이드의 중단 신호 "다음 이미지에 이전 BBox 가 남음"을 막는 방법: 기존 객체를 고쳐 쓰지 않고 `LoadedImage` 를 **새로 만듭니다**.
- `copy.copy(b)` 로 복사본을 편집하므로 `raw_boxes` 는 절대 같이 바뀌지 않습니다.

6.4 편집 함수와 Undo

```
def _push_undo(self) -> None:
    self.cur.undo_stack.append([copy.copy(b) for b in self.cur.boxes])
    del self.cur.undo_stack[:-50]          # 최근 50개만 보관

def add_box(self, cls, x1, y1, x2, y2):
    if cls not in ACTIVE_CLASS_IDS:
        raise ValueError(...)              # Class 4 로 새 BBox 금지
    b = Box(cls, x1, y1, x2, y2).clamp(self.cur.width, self.cur.height)
    if b.w < MIN_BOX_PX or b.h < MIN_BOX_PX:
        return None                        # 실수 드래그 무시
    self._push_undo()
    self.cur.boxes.append(b)
    return b

def replace_box(self, idx, box, push_undo=True) -> bool:
    b = box.clamp(self.cur.width, self.cur.height)
    if b.w < MIN_BOX_PX or b.h < MIN_BOX_PX:
        return False
    if push_undo:
        self._push_undo()
    self.cur.boxes[idx] = b
    return True
```

- Undo 는 "바꾸기 직전 BBox 목록 전체"를 스택에 쌓는 **스냅샷 방식**입니다. 단순하고 실수가 적습니다.

- `replace_box(push_undo=False)` 와 `begin_edit()` 은 **드래그 이동·크기 조절**용입니다. 드래그 중에는 마우스가 움직일 때마다 좌표가 바뀌는데, 그때마다 Undo 를 쌓으면 Ctrl+Z 를 수십 번 눌러야 합니다. 그래서 드래그를 시작할 때 `begin_edit()` 으로 **한 번만** 쌓고, 드래그 중에는 `push_undo=False` 로 바꿉니다.

6.5 save(): 저장 규칙의 전부

```
def save(self, status, scene_type="", note="", issue="", reviewer="") -> str:
    cur, rec = self.cur, self.cur.rec
    work_path = rec.work_label(self.paths)
    if work_path.resolve().is_relative_to(self.paths.raw.resolve()):
        raise RuntimeError("WORK 저장 경로가 RAW 안에 있습니다. 저장 중단.")

    changed_vs_raw = cur.changed_vs_raw()
    if status in ("PENDING", "WORKING", ""):
        status = "EDITED" if changed_vs_raw else "PASS"
    if status == "PASS" and changed_vs_raw:
        status = "EDITED"           # 수정했으면 PASS 가 아니라 EDITED

    if not changed_vs_raw:
        atomic_copy(rec.raw_label(self.paths), work_path)
    else:
        atomic_write_text(work_path, boxes_to_text(cur.boxes, cur.width, cur.height))

    fields = dict(status=status, final_bbox_count=len(cur.boxes), note=note, issue=issue)
    ...
    if status == "REVIEWED":
        fields["reviewer"] = self.worker
    self.manifest.update(rec.rel_image, by=self.worker, **fields)
    self.manifest.save()
    return status
```

한 줄씩 해석

1. **RAW 보호 이중 잠금**: 시작할 때 `check_not_same()` 으로 한 번, 저장 직전에 한 번 더 확인합니다.
2. **상태 자동 보정**: 사람이 실수로 PASS 를 눌러도 라벨이 RAW 와 다르면 EDITED 로 바꿉니다. 그래서 "수정했는데 교차검수에서 빠지는" 일이 없습니다.
3. **안 바꿨으면 원본 복사**: PASS 데이터의 TXT 는 RAW 와 바이트까지 같습니다. 교과 8 에서 "이 파일은 손대지 않았다"를 증명할 수 있습니다.
4. **Manifest 기록**: 누가(`by`), 언제(`updated_at`), 어떤 상태로 저장했는지 남깁니다.
5. 반환값은 **실제로 적용된 상태**입니다. GUI 는 요청한 상태와 다르면 상태바에 "PASS → EDITED 로 저장했습니다"를 띄웁니다.

확인 질문

☐ 라벨을 한 번 수정했다가 Ctrl+Z 로 원래대로 돌린 뒤 PASS 로 저장하면 결과 상태는? (답: PASS. canonical 비교로 RAW 와 같다고 판단)

☐ RAW TXT 에 깨진 줄이 있는 이미지를 그대로 저장하면 상태는? (힌트: `raw_had_errors`)

---

7. validation/validator.py - 자동 검사

```
def check_label_text(rel, text, img_w, img_h) -> list[Issue]:
    issues, boxes = [], []
    for no, line in enumerate(text.splitlines(), start=1):
        if not line.strip():
            continue
        val, err = parse_line(line)
        if err:
            issues.append(Issue("CRITICAL", "PARSE", rel, no, err))
            continue
        cls, cx, cy, w, h = val
        if cls not in VALID_CLASS_IDS:
            issues.append(Issue("CRITICAL", "CLASS_RANGE", ...))
        elif cls in UNUSED_CLASS_IDS:
            issues.append(Issue("REVIEW", "CLASS_4", ...))
        if not all(0 - EPS <= v <= 1 + EPS for v in (cx, cy, w, h)):
            issues.append(Issue("CRITICAL", "COORD_RANGE", ...))
        ...
        for pno, prev in boxes:
            if prev.cls == b.cls and iou(prev, b) >= DUP_IOU_THRESHOLD:
                issues.append(Issue("REVIEW", "DUPLICATE", ...))
        boxes.append((no, b))
    if not boxes and not issues:
        issues.append(Issue("INFO", "EMPTY_LABEL", rel, 0, "BBox 없음 - 정상 이미지인지 사람이 확인"))
    return issues
```

해석

- 심각도를 3단계로 나눴습니다. **CRITICAL** (0 이어야 FINAL 가능) / **REVIEW** (사람 판단 필요) / **INFO** (참고).
- `EPS = 1e-6` 은 부동소수점 오차 허용치입니다. `1.0000001` 같은 값 때문에 멀쩡한 BBox 가 오류로 잡히지 않게 합니다.
- 중복 판단의 **IoU** (Intersection over Union) = 겹친 넓이 ÷ 합친 넓이. 두 BBox 가 완전히 같으면 1, 안 겹치면 0 입니다. 같은 Class 이고 0.9 이상이면 중복으로 의심합니다.
- 빈 TXT 는 오류가 아니라 **INFO** 입니다. 정상 김치 이미지일 수 있기 때문입니다 (가이드 10.1).

```
def image_size(path: Path) -> tuple[int, int, int]:
    with Image.open(path) as im:
        orient = im.getexif().get(274, 1)
        return im.size[0], im.size[1], orient
```

- `Image.open` 은 **헤더만** 읽고 픽셀은 읽지 않습니다. 그래서 900장 4K 를 검사해도 몇 초면 끝납니다.
- EXIF 274 번 태그는 사진 회전 정보입니다. 회전값이 있으면 화면 방향과 YOLO 좌표가 어긋날 수 있어 REVIEW 로 알립니다.

확인 질문

☐ 같은 위치에 Class 2 와 Class 5 BBox 가 겹쳐 있으면 DUPLICATE 일까요? (답: 아니요. 같은 Class 일 때만)

---

8. ui/ - 화면, 마우스, 메뉴

화면 코드는 `src/ui/` 5개 파일과, BBox 를 직접 그리고 고치는 `src/bbox/canvas_editor.py` 로 나뉘어 있습니다. 파일마다 한 가지 일만 하도록 나눠서, 담당자가 자기 부분만 열어 읽을 수 있습니다.

```
app.py            LabelApp (메인 창) - 메뉴 목록, 단축키, 이동 · 저장 · 통계 동작, run()
  ├ panels.py      build_main_window(app) - 위젯 배치만 (수업 예제처럼 Label/Button + grid/pack)
  ├ ../bbox/canvas_editor.py  CanvasEditor - 이미지 · BBox 그리기, 마우스 (LabelApp 이 상속)
  ├ menubar.py     MenuBar - 파일 · 보기 · 도구 · 검수 · 통계 · 도움말 메뉴
  ├ dialogs.py     Validation 결과 표, 진행현황, Class 통계, 단축키 안내
  └ style.py       색 · 글꼴 · 상태 기호, tool_button / action_button / big_button / group_box
```

8.1 화면 구성 (`panels.build_main_window`)

위치	들어 있는 것	만드는 함수
맨 위	메뉴 막대: 파일 · 보기 · 도구 · 검수 · 통계 · 도움말	`app.build_menu()` → `MenuBar`
왼쪽	이미지 목록 (◯ ✓ ✎ ! ✔), 보기 필터	`build_image_list()`
가운데	파일명 (n/N), 출처·크기, Canvas (이미지 + BBox)	`build_center()`
오른쪽 1열	Class 선택, BBox 정보 (X·Y·너비·높이 입력), BBox 목록	`build_class_panel()`
오른쪽 2열	작업 상태, Scene Type, 작업자/검수자, 파일 정보, 이슈/메모	`build_status_panel()`
아래 1	⚠ 경고 줄 (전체 너비)	`build_main_window()`
아래 2	보기 도구 · 라벨 도구 · 이전/다음 · [저장] [저장 후 다음]	`build_bottom_tools()`
맨 아래	상태바 (진행 현황, 메시지)	`build_main_window()`

```
def build_main_window(app):
    root = app.root
    # 아래쪽(상태바, 도구 막대)을 먼저 붙여야 창을 줄여도 버튼이 가려지지 않는다.
    app.status_bar = tk.StringVar(value="준비")
    status = tk.Label(root, textvariable=app.status_bar, anchor="w", relief=tk.SUNKEN, ...)
    status.pack(side=tk.BOTTOM, fill=tk.X)

    build_bottom_tools(app, root)
    ...                                  # 경고 줄도 BOTTOM 으로
    panes = tk.PanedWindow(root, orient=tk.HORIZONTAL, ...)
    panes.pack(fill=tk.BOTH, expand=True, ...)
```

- 이 파일은 **배치만** 합니다. 버튼을 눌렀을 때의 동작은 `command=app.save` 처럼 app 의 메서드에 맡깁니다. 그래서 "어디에 무엇이 있는지"와 "무엇을 하는지"가 섞이지 않습니다.
- `tk.PanedWindow` 로 왼쪽/가운데/오른쪽을 나눠서, 경계선을 끌어 폭을 조절할 수 있습니다.
- pack 순서가 중요합니다. **상태바 → 도구 막대 → 경고 줄을 먼저 `side=BOTTOM` 으로 붙이고**, 그다음 본문을 `expand=True` 로 붙여야 창이 작아져도 버튼이 사라지지 않습니다.
- 경고 줄을 이미지 바로 아래가 아니라 **전체 너비 줄**로 둔 이유: 그래야 이미지 목록·이미지·오른쪽 칸의 아래쪽 선이 한 줄로 맞습니다.

- 위젯은 수업 예제와 같은 `tk.Label`, `tk.Button`, `tk.Entry`, `StringVar` + `grid` / `pack` 이고, 선택 칸만 `ttk.Combobox` 를 씁니다.

8.2 도구 버튼 색 (`style.action_button`)

```
def highlight_only(button, group):
    """group 안에서 button 하나만 파란색, 나머지는 회색."""
    for other in group:
        paint_active(other, other is button)


def action_button(parent, text, command, group):
    button = tool_button(parent, text, None)
    group.append(button)

    def run():
        highlight_only(button, group)
        button.update_idletasks()          # Validation 처럼 오래 걸려도 파란색이 먼저 보이게
        command()

    button.configure(command=run)
    return button
```

- 아래 도구 막대 10개 버튼이 같은 `group` 리스트를 공유합니다. 누른 버튼 하나만 파란색이 됩니다.
- 버튼의 원래 동작(`command`)을 `run()` 이라는 **안쪽 함수로 감싸서** "색칠 → 원래 동작" 순서로 실행합니다. 원래 함수는 하나도 고치지 않았습니다 (8.6 의 `guard` 와 같은 클로저 패턴).
- 키보드 `W`, `V` 로 모드를 바꿀 때도 `set_mode()` 가 `highlight_only()` 를 불러 색이 맞춰집니다.

8.3 이미지 그리기 (`canvas_editor.render`) - 4K 를 빠르게

```
def render(self):
    ...
    rect = self.view.visible_image_rect(c.winfo_width(), c.winfo_height())   # 보이는 원본 영역만
    if rect:
        x1, y1, x2, y2 = rect
        level = 0
        while level + 1 < len(self.pyramid) and self.view.scale <= 0.5 ** (level + 1):
            level += 1                                  # 축소 배율이면 작은 이미지 사용
        f = 2 ** level
        src = self.pyramid[level]
        crop = src.crop((x1 // f, y1 // f, ...))
        ...
        self.photo = ImageTk.PhotoImage(crop.resize((dw, dh), resample))
        c.create_image(px, py, image=self.photo, anchor="nw")
    self.draw_boxes()
```

최적화 두 가지

1. **보이는 부분만 자르기**: 8배 확대하면 화면에는 4K 의 일부만 보입니다. 그 부분만 잘라 확대하므로 빠릅니다.

2. **이미지 피라미드**: `make_pyramid()` 에서 원본, 1/2, 1/4 크기를 미리 만들어 둡니다(`reduce(2)`). 30% 배율로 볼 때는 1/4 크기를 쓰니 처리할 픽셀이 16배 적습니다.

`self.photo` 에 저장하는 이유: Tkinter 는 이미지 객체를 Python 변수가 잡고 있지 않으면 **사라집니다**(가비지 컬렉션). 지역 변수로만 두면 화면이 하얗게 나옵니다. Tkinter 초보가 가장 많이 겪는 버그입니다.

8.4 마우스: 모드 + drag 딕셔너리 (`canvas_editor`)

수업 예제 3(`app3.py`)의 `on_mouse_down / on_mouse_drag / on_mouse_up` 과 같은 구조입니다. 다만 "그리기" 하나가 아니라 **그리기·이동·크기 조절** 세 가지를 구분해야 해서 `drag` 딕셔너리를 씁니다.

```
def on_left_down(self, event):
    if self.menubar.is_open():              # 메뉴를 닫으려는 클릭은 그리기로 쓰지 않음
        self.menubar.close()
        return
    ...
    ix, iy = self.view.to_image(event.x, event.y)
    handle = self.hit_handle(event.x, event.y)
    if handle:                              # 핸들을 잡았으면 크기 조절
        box = self.s.cur.boxes[self.selected]
        self.drag = dict(kind="resize", h=handle, orig=box.normalized(),
                         pushed=False, sx=event.x, sy=event.y)
        return
    if self.mode == "select":               # 선택 이동 모드
        hit = self.s.find_box_at(ix, iy)
        self.selected = hit
        if hit is not None:
            self.select_class_of(hit)
            self.drag = dict(kind="move", orig=..., pushed=False, ix=ix, iy=iy, sx=event.x, sy=event.y)
        ...
        return
    self.drag = dict(kind="draw", ix=ix, iy=iy, sx=event.x, sy=event.y)   # 새 BBox 모드
```

- 마우스를 누르는 순간 **무엇을 할지 결정**하고, 그 결정을 `self.drag` 딕셔너리에 담아 둡니다. `kind` 는 `draw` / `move` / `resize` 중 하나입니다.
- 이후 `on_left_drag` 와 `on_left_up` 은 `drag["kind"]` 만 보고 동작합니다. 이런 방식을 **상태 기계(state machine)** 라고 부릅니다.
- `orig` 에 드래그 시작 시점의 BBox 를 저장해 두고, 매번 **원래 BBox + 마우스 이동량**으로 새로 계산합니다. 이동량을 누적하지 않으므로 오차가 쌓이지 않습니다.

```
def on_left_drag(self, event):
    ...
    if not d["pushed"]:
        if abs(event.x - d["sx"]) < DRAG_START_PX and abs(event.y - d["sy"]) < DRAG_START_PX:
            return                          # 3px 미만 움직임은 클릭으로 간주
        self.s.begin_edit()                 # 진짜 드래그가 시작될 때 Undo 1번
        d["pushed"] = True
    o = d["orig"]
    ix, iy = self.view.to_image(event.x, event.y)
    if d["kind"] == "move":
        dx = max(-o.x1, min(cur.width - o.x2, ix - d["ix"]))   # 이미지 밖으로 못 나가게
```

```
            dy = max(-o.y1, min(cur.height - o.y2, iy - d["iy"]))
            new_box = Box(o.cls, o.x1 + dx, o.y1 + dy, o.x2 + dx, o.y2 + dy)
        else:                               # resize: 잡은 핸들 방향의 변만 움직임
            x1, y1, x2, y2 = o.x1, o.y1, o.x2, o.y2
            if "w" in d["h"]: x1 = ix
            if "e" in d["h"]: x2 = ix
            if "n" in d["h"]: y1 = iy
            if "s" in d["h"]: y2 = iy
            new_box = Box(o.cls, x1, y1, x2, y2)
        self.s.replace_box(self.selected, new_box, push_undo=False)
        self.draw_boxes()
```

- 핸들 이름을 방위(북 n, 남 s, 동 e, 서 w)로 지은 덕분에 `"w" in h` 같은 문자열 검사 4줄로 **8방향 크기 조절**이 모두 처리됩니다. `'nw'` 핸들은 `w` 와 `n` 이 둘 다 들어 있으니 x1 과 y1 이 함께 움직입니다.
- 이동할 때 `dx` 를 `[-x1, 이미지폭-x2]` 범위로 가둬서, BBox 가 크기는 유지한 채 이미지 가장자리에서 멈춥니다.
- 크기 조절 중 반대편을 넘어가도(x1 > x2) 놓는 순간 `normalized()` 로 정리합니다.

8.5 저장 안 한 변경 보호 (`app.confirm_leave`)

```
def confirm_leave(self):
    if not self.s.is_dirty():
        return True
    answer = messagebox.askyesnocancel("저장하지 않은 변경", "...저장할까요?...")
    if answer is None:
        return False          # 취소 -> 머무르기
    if answer:
        return self.save()    # 예 -> 저장 성공해야 이동
    return True               # 아니오 -> 버리고 이동
```

- 이전/다음, 목록 클릭, 필터 변경, 폴더 열기, 창 닫기, 터미널 Ctrl+C 가 **모두 이 함수 하나**를 거칩니다. 그래서 어떤 경로로 떠나든 저장 유실을 막을 수 있습니다.
- "예"를 눌러도 저장이 실패하면(`save()` 가 False) 이동하지 않습니다.

8.6 단축키 보호 (`app.bind_keys` 의 `guard`)

```
def guard(func):
    """글자를 입력하는 칸(Entry, 콤보, 메모)에서는 단축키를 무시한다."""
    def handler(event):
        if isinstance(event.widget, (tk.Entry, ttk.Entry, tk.Text)):
            return None       # 글자 입력 칸에서는 단축키 무시 (ttk.Combobox 도 ttk.Entry 의 자식)
        func()
        return "break"        # 다른 바인딩으로 이벤트가 퍼지지 않게
    return handler
```

- 함수를 받아 함수를 돌려주는 **클로저/데코레이터 패턴**입니다.
- 메모 칸에 "pass" 라고 타이핑했는데 P 단축키가 눌려 상태가 바뀌면 큰일이므로, 입력 칸에서는 단축키를 끕니다.
- `Ctrl+S` 등은 Caps Lock 이 켜져 있으면 대문자(`<Control-S>`)로 들어와서 **소문자·대문자 둘 다** 묶었습니다.
- 모든 버튼은 `takefocus=0` 입니다. 버튼이 키보드 포커스를 가져가면 Space 가 "마지막으로 누른 버튼"을 다시 누르는 Tkinter 기본 동작 때문에 엉뚱한 버튼이 눌립니다.

8.7 메뉴 막대 (`menubar.py`) - 왜 tk.Menu 를 안 썼나

WSL(WSLg) 화면에서 `tk.Menu` 를 열고 다른 메뉴로 옮기면 **이전 드롭다운이 잔상으로 남았습니다.** tk.Menu 의 드롭다운은 메인 창과 별개인 작은 창이고, WSLg 가 그 창이 닫힌 자리를 제때 다시 그리지 않기 때문입니다.

```
class Dropdown(tk.Frame):
    """드롭다운 하나. 메인 창 안에 place() 로 띄우는 Frame."""
    ...
    def _place(self, x, y):
        ...
        self.place(x=x, y=y)
        self.lift()            # 다른 위젯보다 위에 보이게
```

- 해결: 드롭다운을 **별도 창이 아니라 메인 창 안의 Frame** 으로 만들어 `place()` 로 띄우고, 닫을 때 `destroy()` 합니다. 메인 창이 직접 다시 그리므로 잔상이 생길 수 없습니다.
- 메뉴는 항상 하나만 열립니다. 열린 상태에서 옆 제목에 마우스를 올리면 이전 것을 닫고 새 것을 엽니다 (`_on_title_enter`).
- 메뉴가 열려 있을 때 바깥을 누르면 **메뉴만 닫히고 그 클릭은 무시**됩니다 (`CLICK_GUARD` bindtag). 메뉴를 닫으려다 이미지 목록이 바뀌거나 버튼이 눌리는 일을 막습니다.
- `Alt+F/V/T/Q/S/H` , `F10` 으로 열고 방향키·Enter·Esc 로 움직일 수 있습니다. 메뉴가 열린 동안 키보드는 메뉴 막대가 받도록 `bindtags` 에서 root 태그를 뺐습니다 (A, D, Space 단축키가 같이 눌리지 않게).

8.8 터미널 Ctrl+C 처리 (`app.run`)

```
# Ctrl+C 도 저장 확인을 거쳐 종료. mainloop 중에는 Python 이 시그널을 못 받으니 주기적으로 깨운다.
signal.signal(signal.SIGINT, lambda *args: root.after(0, app.on_close))

def tick():
    root.after(250, tick)

tick()
```

- Ctrl+C 는 운영체제가 보내는 SIGINT 신호입니다. 이 신호를 받으면 `on_close()` (저장 확인 후 종료)를 실행하도록 등록했습니다.
- Tkinter 의 `mainloop()` 가 C 코드 안에서 기다리는 동안 Python 은 신호를 확인하지 못합니다. 그래서 0.25초마다 아무것도 안 하는 `tick` 을 실행해 Python 이 깨어나 신호를 확인하게 합니다.

확인 질문

☐ `self.photo = ...` 대신 `photo = ...` 로 쓰면 어떤 현상이 생길까요?

☐ 선택 이동 모드에서 BBox 를 클릭만 하고 움직이지 않으면 Undo 스택에 쌓일까요? (답: 아니요. `pushed` 가 False 로 남음)

☐ 드롭다운을 `tk.Toplevel` 로 만들면 왜 WSL 에서 잔상이 생길 수 있을까요?

☐ `panels.py` 에는 왜 `if` , `for` 같은 동작 코드가 거의 없을까요?

---

9. scripts - 운영 도구 00~10

스크립트는 `.venv` 를 켠 뒤 프로젝트 폴더에서 실행합니다: `cd ~/kimchi_labeler && source .venv/bin/activate`

스크립트	핵심 코드 아이디어
`00_raw_snapshot.py`	모든 RAW 파일의 `hashlib.sha256` 을 CSV 로 저장 → `--verify` 때 다시 계산해 비교. 1바이트만 달라도 해시가 완전히 달라집니다
`01_build_manifest.py`	`scan_images` + `add_records` → 기존 행은 건드리지 않고 새 이미지만 PENDING 추가
`02_validate.py`	`validate()` 결과를 CSV 로 저장, CRITICAL 이 있으면 **종료코드 1** (다른 스크립트·CI 가 실패를 알 수 있게)
`03_assign.py`	Dataset/split 그룹마다 "지금 가장 적게 받은 사람"에게 한 장씩 → 모두 골고루. 검수자 = 다음 사람 `workers[(i+1) % n]`
`04_merge.py`	`merge_manifests` + 행을 가져온 폴더에서 라벨 복사 + 담당·검수자 외 수정 / REVIEWED 후 재수정 감지
`05_qa_summary.py`	`stats.collect` 로 수량·Class·보정 내역·상태·Validation 을 Markdown 표로 → `reports/qa_summary.md`
`06_pass_sample.py`	PASS 중 절반은 위험도 순(작은 BBox, 많은 BBox, 희소 Class), 절반은 무작위
`07_build_final.py`	조건 3개(상태·CRITICAL·RAW 해시) 중 하나라도 실패하면 **만들지 않음**. `final/images`, `final/labels` 에 같은 파일명으로 복사하고 원래 Dataset/split 은 `final_file_map.csv` 에 남김. 기존 FINAL 은 `_old_날짜` 로 보관
`08_handoff.py`	`05` 의 build 함수를 재사용해 FINAL 기준 Handoff 문서 생성 → `docs/subject08_handoff.md`
`09_golden_test.py`	임시 WORK 에서 왕복 오차, 무수정 저장=RAW, 편집 후 재로드, RAW 보존을 이미지마다 확인
`10_export_manifest.py`	작업 manifest 를 제출 형식(`manifests/dataset_manifest.csv`)으로. 라벨이 RAW 와 같으면 DONE, 다르면 EDITED

9.1 03_assign.py 의 균등 배분

```
for key in sorted(groups):                # (dataset, split) 그룹마다
    rows = groups[key]
    rng.shuffle(rows)
    for r in rows:
        w = min(workers, key=lambda x: (load[x], workers.index(x)))   # 가장 적게 받은 사람
        r["assignee"], r["reviewer"] = w, reviewer_of[w]
        load[w] += 1
```

- `min(..., key=...)` 는 "key 값이 가장 작은 원소"를 돌려줍니다. key 가 `(받은 수, 순서)` 튜플이라 받은 수가 같으면 앞 사람이 먼저 받습니다.
- 그룹별로 돌기 때문에 validation 데이터가 한 사람에게 몰리지 않습니다.

9.2 stats.py 의 보정 내역 계산 (diff_boxes)

```
pairs = sorted(((iou(r, f), i, j) for i, r in enumerate(raw) for j, f in enumerate(final)), reverse=True)
for v, i, j in pairs:
    if v < MATCH_IOU or i in used_r or j in used_f:
        continue
    used_r.add(i); used_f.add(j)
```

```
    if raw[i].cls != final[j].cls:   out["class_changed"] += 1
    elif v < MOVED_IOU:              out["moved"] += 1
    else:                            out["same"] += 1
out["deleted"] = len(raw) - len(used_r)
out["added"]   = len(final) - len(used_f)
```

- RAW BBox 와 최종 BBox 의 모든 쌍을 IoU 가 큰 순서로 정렬해, 큰 것부터 짝을 지어 줍니다(**그리디 매칭**).
- 짝이 생긴 것은 유지/위치수정/Class변경으로, 짝이 없는 RAW 는 삭제, 짝이 없는 최종은 추가로 셉니다. QA Summary 의 "라벨 보정 내역" 표가 이렇게 만들어집니다.

---

10. tests - 자동 테스트

`tests/test_core.py` 는 4개 테스트로 구성됩니다.

테스트	확인하는 것
`test_yolo_roundtrip`	파싱 → 다시 쓰기가 원문과 같은지, 깨진 줄 3종을 오류로 잡는지
`test_view_zoom_keeps_point`	Zoom 후에도 마우스 아래 점이 그대로인지, 화면↔원본 왕복
`test_session_rules`	실수 드래그 무시, 이미지 밖 Clamp, Class 4 금지, Undo, 이미지 전환 시 초기화, PASS→EDITED
`test_full_workflow`	가짜 데이터 생성 → Manifest → 배분 → 2명 작업 → 병합 → 교차검수 → PASS 표본 → FINAL → Handoff → **RAW 무변경**

- 마지막 테스트가 **프로젝트 전체 Pipeline 을 1분 안에 재현**합니다. 코드를 고친 뒤 이것만 통과하면 큰 사고는 없다고 볼 수 있습니다.
- 규칙: 코드를 고치면 `python tests/test_core.py` 와 `python scripts/09_golden_test.py` 를 **반드시** 다시 실행합니다.

---

11. 따라가 보기: Space 를 눌렀을 때 일어나는 일

```
① 키보드 Space
② app.bind_keys 의 guard → 입력 칸이 아니면 save_next() 호출
③ save_next() → save()
④ save():
    - REVIEWED 인데 내 작업이면 경고
    - Class 4 가 남아 있으면 REVIEW 로 바꿈
    - 메모 칸 여러 줄을 " / " 로 합침
    - session.save(상태, scene, note, issue, reviewer)
        ├ RAW 경로 검사
        ├ changed_vs_raw() 로 PASS→EDITED 보정
        ├ 안 바꿈: atomic_copy(RAW→WORK) / 바꿈: atomic_write_text
        └ manifest.update(...) → manifest.save() (이것도 atomic)
    - load_meta(), update_list_item() → 왼쪽 목록 기호/색 갱신
⑤ 성공하면 goto(index + 1, check_dirty=False)
    ├ session.load() → 새 LoadedImage (이전 BBox 버림)
```

```
    ├ Image.open + make_pyramid()          (실패하면 show_empty_view() 로 화면을 비움)
    └ fit() → render() → draw_boxes()      (canvas_editor.py)
```

이 흐름을 말로 설명할 수 있으면 프로그램 구조를 이해한 것입니다. 발표 시연 때 이 순서로 설명하면 좋습니다.

---

12. 연습 문제

난이도 ★ ~ ★★★. 실제 코드를 바꿔 보는 연습이므로 **Feature Freeze 이후에는 별도 Git 브랜치**에서만 하세요.

1. ☐ ★ `configs/classes.yaml` 에서 Class 1 의 `color` 를 바꿔 보세요. 화면 어디어디가 바뀌나요?
2. ☐ ★ `MIN_BOX_PX` 를 3 → 10 으로 바꾸면 어떤 BBox 가 만들어지지 않을까요? 다시 3 으로 되돌리세요.
3. ☐ ★ `style.py` 의 `BLUE` 색을 바꿔 보세요. 어떤 버튼들이 같이 바뀌나요? (`paint_active` 를 쓰는 곳)
4. ☐ ★★ 선택된 BBox 를 방향키로 1px 씩 움직이는 기능을 추가해 보세요. 힌트: `app.bind_keys` 에 `<Up>` 등을 묶고, `self.s.replace_box(self.selected, Box(...))` 사용. 지금 `←` `→` 는 이전/다음이니 충돌을 어떻게 피할지 정하세요.
5. ☐ ★★ `validator.py` 에 "이미지 면적의 50% 넘는 너무 큰 BBox" 를 REVIEW 로 잡는 규칙을 추가해 보세요.
6. ☐ ★★ 왼쪽 이미지 목록 위에 파일명 검색 칸을 추가해 보세요. 위젯은 `panels.build_image_list`, 동작은 `self.s.view_keys` 를 걸러서 `app.refresh_image_list()`.
7. ☐ ★★★ `tests/test_core.py` 에 "BBox 이동 후 저장하면 EDITED, Undo 후 저장하면 PASS" 를 확인하는 테스트를 추가해 보세요.

---

13. 발표 예상 질문과 답

질문	답의 핵심	근거 코드
RAW 를 안 건드렸다는 걸 어떻게 증명하나요?	시작 때 SHA-256 해시 기록, 매일·FINAL 전에 재검증. 저장 경로도 이중 검사	`00_raw_snapshot.py`, `session.save()`
Zoom 하면 BBox 가 어긋나지 않나요?	BBox 는 원본 좌표로만 저장, 화면 변환은 그릴 때만. 왕복 오차 1e-6 이하 시험	`view.py`, `09_golden_test.py`
수정했는데 PASS 로 저장하면요?	프로그램이 RAW 와 비교해 자동으로 EDITED 로 바꾸고, EDITED 는 100% 교차검수	`session.save()`
6명이 동시에 작업하면 충돌은요?	담당자 배분 + updated_at 기준 병합 + 담당·검수자 외 수정과 검수 후 재수정 자동 보고	`03_assign.py`, `04_merge.py`
Class 4 는 왜 지우지 않았나요?	미사용 Class 지만 실제 이물일 수 있어 사람이 판정. 자동 삭제는 정보 손실	`classes.yaml` 의 `enabled: false`, `validator`
빈 TXT 는 오류 아닌가요?	정상 김치(Negative)일 수 있어 INFO 로만 알리고 사람이 확인	`validator.check_label_text()`
4K 이미지인데 느리지 않나요?	보이는 영역만 자르기 + 1/2·1/4 피라미드, Validation 은 헤더만 읽기	`canvas_editor.render()`, `validator.image_size()`
저장 중 꺼지면요?	임시파일 → os.replace 로 원자적 교체. 깨진 TXT 가 남지 않음	`yolo_io.atomic_write_text()`

질문	답의 핵심	근거 코드
메뉴는 왜 직접 만들었나요?	WSLg 에서 tk.Menu 드롭다운 잔상. 메인 창 안 Frame 으로 띄워 해결	`ui/menubar.py`
프로그램이 맞게 동작하는지 어떻게 아나요?	자동 테스트가 전체 Pipeline 재현, Golden Sample, Pilot, Acceptance Test	`tests/test_core.py`

> 본 프로젝트는 AI캠퍼스 교육을 위해 실제 산업데이터와 유사한 분포 구조로 100% 가상 생성된 조각김치 이물검출 학습데이터를 활용하여 수행하였습니다.